TECHNICAL REPORT

チームの仕事を見える化する業務管理ツール「HAKOBI」の設計と実装

日々の記録を、状況確認と定例・週報に生かす

> **個人用：1 人で使うデスクトップ版　／　チーム用：社内 LAN のサーバーで共有**

作成日　2026 年 10 月 17 日

対象　2026 年 10 月 17 日時点の HAKOBI（デスクトップ版 1.0.5、コミット f9c1bc6 に 8.2 節の修正を加えたもの）

概要

係やグループで仕事を進めるとき、ほかの人の仕事がどこまで進んでいるかは、本人に聞かないと分からないことがある。定例や週報の前には、記憶をたどって別の一覧を作り、報告のたびに同じ内容を書き直す手間もかかる。この手間を減らし、チームの予定・担当・進捗を同じ画面で確かめられるようにするため、業務管理ツール HAKOBI を開発した。本レポートでは、2026 年 10 月 17 日時点の実装について、設計と実装の内容、品質の確認結果を報告する。

HAKOBI はガントチャートを中心とした画面を持ち、作業の区切りに残した進捗率と進捗ログを、チームの状況確認、定例での相談、週報の下書きに使い回す。1 人で使う個人用は Electron で作ったデスクトップアプリとして動く。チーム用は社内 LAN（Local Area Network）に置いたサーバーへ、ブラウザとデスクトップ版から接続する。両者は同じ React の画面を共有し、データを読み書きする窓口だけを切り替える。チーム用には WebSocket による変更の通知、定例のための報告タブと定例モード、Microsoft 365 Copilot へ渡せる Markdown 形式の出力を実装した。

2026 年 3 月に個人用として開発を始め、5 月にチーム用を加えた。10 月 17 日に型チェック 4 種と Web のビルドを実行し、すべて成功した。自動テストは、最初の実行で 1 コマンドが時間切れとなった。アプリの定期処理のタイマーがテストのプロセスの終了を妨げていたためで、これを直したあとは 15 コマンドすべてが合格した。10 月 9 日の全体確認で見つけた 13 件の不具合は修正済みである。これらの結果から、目指した五つの姿（チームの仕事が見える、記録が一度で済むなど）に対応する機能を実装できたと判断した。導入による効果はまだ測っていない。今後は定例と週報にかかる時間などを測り、効果を確かめる。

目次

1. 序論 5
1.1 背景 5
1.2 HAKOBI の位置付け 5
1.3 目的と HAKOBI が目指した姿 5
1.4 対象範囲 6
1.5 本レポートの構成 6
2. 開発の歩み 7
2.1 全体の流れ 7
2.2 1 人で使うデスクトップアプリ（3〜4 月） 7
2.3 チーム用サーバーの追加（5〜6 月） 7
2.4 進捗の共有と見やすさ（6〜9 月） 8
2.5 チーム接続のデスクトップ版と定例の改善（10 月） 8
3. 設計の考え方と全体構成 8
3.1 五つの設計方針 8
3.2 構成要素 9
3.3 データの流れと同期 10
3.4 データの構造 11
3.5 技術の選択 11
4. チームの仕事を見える化する仕組み 12
4.1 ガントチャート 12
4.2 依存関係の再計算 13
4.3 チーム画面 14
4.4 進捗ログと相談 14
5. 記録を定例と週報に使う仕組み 15
5.1 書く場所の役割と記録の使い回し 15
5.2 報告タブ 16
5.3 定例モード 16
5.4 週報の下書き 17
5.5 進捗レポートの数え方 17
6. 一人ひとりの仕事を進める機能 18
6.1 今日の計画と作業時間 18
6.2 繰り返す仕事と ToDo リスト 18
6.3 検索・移動と動作の軽さ 19
6.4 デスクトップ版のチーム接続 19
7. 運用と安全の仕組み 20

7.1 アカウントと認証 20
7.2 社内 LAN での運用 20
7.3 バックアップ 20
7.4 デスクトップ版の配布 20
7.5 個人用データの取り込み 21
8. 品質の確認 21
8.1 確認の方法 21
8.2 結果 21
8.3 全体確認で見つけた不具合と対策 22
9. 考察 23
9.1 目指した姿ごとの到達点 23
9.2 画面を共通にしたことの利点と代償 24
9.3 SQLite と 1 台のサーバーの利点と限界 24
9.4 安全性についての判断 24
9.5 効果の確かめ方 25
10. 結論と今後の展望 25
10.1 結論 25
10.2 業務への貢献 26
10.3 今後の展望 26
参考文献 27
内部資料 27
外部資料 27
付録 A 用語と略語 28
付録 B 主な既定値 29
付録 C 週報用エージェントの指示の例 30

1. 序論

1.1 背景

係やグループの仕事は、複数のメンバーが期限のある業務を分担して進める。メンバーの仕事の状況は本人に聞かないと分からないことがある。定例や週報の前には、期間中に何が進んだかを思い出しながら別の一覧を作る。報告のたびに、同じ内容を一から書き直すこともある。

こうした手間は、仕事の記録が報告の形で残っていないことから生じる。作業の区切りに短い記録を残し、それを確認や報告にそのまま使えれば、聞き直しや書き直しを減らせる。 HAKOBI はこの考え方をチームの日常業務に取り入れるために開発した。

本レポートでいうガントチャートは、仕事を横棒で時間軸に並べ、開始日、期限、進み具合を示す図である。進捗ログは、仕事ごとに日付つきで積み上がる短い報告を指す。

1.2 HAKOBI の位置付け

HAKOBI は作業の区切りに残す記録を 1 か所に集め、チームの状況確認、定例、週報の準備に使う道具として位置付ける。使い方は二つあり、本文では 1 人で使う形を「個人用」、社内 LAN（Local Area Network）のサーバーを複数人で使う形を「チーム用」と呼ぶ。表 1 に違いを示す。チーム用にはブラウザから使う方法と、デスクトップ版から接続する方法（チーム接続）がある。

表 1 個人用とチーム用の違い

比較する項目	個人用	チーム用
利用する画面	デスクトップ版	ブラウザ、またはデスクトップ版のチーム接続
データの保存先	自分の PC（Personal Computer）内	社内 LAN のサーバー
主な使い方	自分の仕事と時間を整理する	担当を分け、進捗と相談を共有する
メンバーの管理	不要	管理者がアカウントを発行する
想定する規模	1 人	1 つのサーバーで 6 人程度のチーム[I1]

1.3 目的と HAKOBI が目指した姿

本レポートでは HAKOBI の開発目標と、そのために実装した機能や仕組みを説明する。あわせて、品質の確認結果と、設計上の判断の利点と限界を示す。

開発目標を表 2 の五つに整理した。G1 と G2 はチームの課題に対応する目標、G3 は一人ひとりの日常業務の目標、G4 と G5 は使い始めやすさと運用の目標である。以降の章では、各機能がどの目標に対応するかを示す。

表 2 HAKOBI が目指した五つの姿

記号	目指した姿	具体的に実現したいこと	主な説明
G1	チームの仕事が見える	だれが何を進めていて、どの仕事の期限が近いかを同じ画面で確かめる	4 章
G2	記録が一度で済む	作業の区切りに残した記録を、定例と週報にそのまま使う	5 章
G3	自分の仕事を進めやすい	今日の計画、作業時間、繰り返す点検を一つの道具で扱う	6 章
G4	個人でもチームでも同じ画面	1 人で使い始め、チームで使うときも操作を覚え直さない	3 章
G5	社内で軽く運用できる	社内 LAN の中で完結し、アカウント、バックアップ、配布を管理者が扱える	7 章

1.4 対象範囲

対象は個人用のデスクトップ版、チーム接続のデスクトップ版、ブラウザ版の画面、チーム用のサーバー、自動テストおよび運用文書である。内容は 2026 年 10 月 17 日時点の実装に基づく。

導入による効果（時間の短縮など）は、まだ測っていないため扱わない。インターネットへの公開、サーバーに接続できない間の編集と自動同期、複数のグループの同時利用、デスクトップ版の自動更新は、実装の範囲外である [I3]。利用者の PC やサーバーの OS（Operating System）が侵害された場合の対策も対象外とする。

1.5 本レポートの構成

第 2 章で開発の経緯、第 3 章で設計方針と全体構成を示す。第 4 章ではチームの仕事を見える化する仕組み、第 5 章では記録を定例と週報に使う仕組み、第 6 章では一人ひとりの仕事を進める機能を説明する。第 7 章で運用と安全の仕組みを紹介する。第 8 章に品質確認の方法と結果を示し、第 9 章で目指した姿の到達点と設計上の判断を考察する。第 10 章では結論と今後の課題を述べる。

2. 開発の歩み

2.1 全体の流れ

2026 年 3 月 4 日にリポジトリを作り、約 7 か月で現在の構成となった。図 1 に主な節目を示す。2026 年 10 月 17 日までの変更は 159 コミットで、main の履歴に番号が残るプルリクエスト（Pull Request、PR）は 87 件である[I5]。開発では機能を小さく分けて実装し、使って気付いた点をすぐに直す流れを繰り返した。



図 1 HAKOBI の開発の歩み（2026 年）

2.2 1 人で使うデスクトップアプリ（3〜4 月）

3 月 10 日に、Electron のデスクトップアプリとして開発を始めた。3 月 24 日にガントチャート、今日の計画、概要の画面を加えた。4 月上旬には、依存関係の編集、タスクの説明とメモの分離、ガント上での並べ替え、明暗の表示の切り替えを実装した[I5]。

2.3 チーム用サーバーの追加（5〜6 月）

5 月 30 日に、複数人で使う Web 版とサーバーを加えた。このとき、画面がデータを読み書きする窓口を、プロセス間通信（Inter-Process Communication、IPC）から HTTP（Hypertext Transfer Protocol）と WebSocket へ切り替えた。窓口だけを切り替えたため、React の画面はほぼそのまま使えた。担当者の割り当て、チーム画面、管理者によるアカウントの発行、リアルタイムの同期もこのときに実装した。

6 月 11 日には、個人用データの取り込み、副担当、Windows サービスとしての起動、設定ファイルによる運用を加えた[I5]。

2.4 進捗の共有と見やすさ（6〜9 月）

6 月から 9 月にかけて、進捗ログへの返信、通知、カンバン、いいね、タスク別のレポートを順に加えた。9 月 25 日には、定例で使う報告タブと、ガントの見やすさの改善を取り込んだ[I5]。

2.5 チーム接続のデスクトップ版と定例の改善（10 月）

10 月 9 日に、デスクトップ版からチームのサーバーへ接続できるようにした。同日の全体確認では 13 件の不具合を見つけ、修正して配布物を 1.0.1 に更新した[I4]。10 月 13 日には、OS の通知と未読バッジ、サーバーからのインストーラーの配布、新しいバージョンのお知らせを加えた（デスクトップ版 1.0.5）。10 月 14 日から 17 日にかけては、ToDo リスト、定例モード、報告のリマインド、移動・操作のパレット、管理者向けの機能、ライトテーマの作り直しを実装した[I5]。

3. 設計の考え方と全体構成

3.1 五つの設計方針

設計では次の五つを方針とした[I2]。

一つ目はガントチャートを中心にすることである。日程、担当、進捗、仕事の順序を一つの画面で確かめられるようにし、カンバンや報告などの画面は同じタスクを別の切り口で見せる。

二つ目は個人用とチーム用で画面を共通にすることである。両者は同じ React の画面を使う。画面はデータベース（Database、DB）を直接扱わず、`window.api` という窓口を通して読み書きする。窓口の中身だけを、個人用では IPC、チーム用では HTTP と WebSocket の実装に切り替える。

三つ目は記録の役割を分けることである。タスクの「説明」には目的、「メモ」には今の状況を書き、「進捗ログ」には日付つきの報告を積み上げる（5.1 節）。役割が決まっていると、同じ記録を確認にも報告にも使える。

四つ目は社内 LAN の中で、少ない手間で運用することである。サーバーは 1 つのプロセスで動き、データは SQLite のファイルに保存する。別の DB 製品を構築する必要はない。

五つ目はすでに使っている人が使い続けられるようにすることである。チーム用を加えた後も、個人用のデスクトップ版は動き続けるように保つ。サーバーが配信する画面とインストール済みのデスクトップ版は別々に更新されるため、新旧が混ざっても動くようにした（3.3 節）。

3.2 構成要素

HAKOBI の構成を図 2 に示す。個人用では、デスクトップ版の中の画面が IPC で main プロセス（Electron のアプリの入口となるプロセス[E1]）を呼び、main プロセスが PC 内の SQLite を読み書きする。チーム用では、ブラウザとデスクトップ版の画面が同じサーバーへ接続する。デスクトップ版は、サーバーが配信する画面を表示したうえで、タイマー、通知、トレイなどの端末の機能を加える。デスクトップ版がサーバーの DB ファイルへ直接接続することはない[I1]。

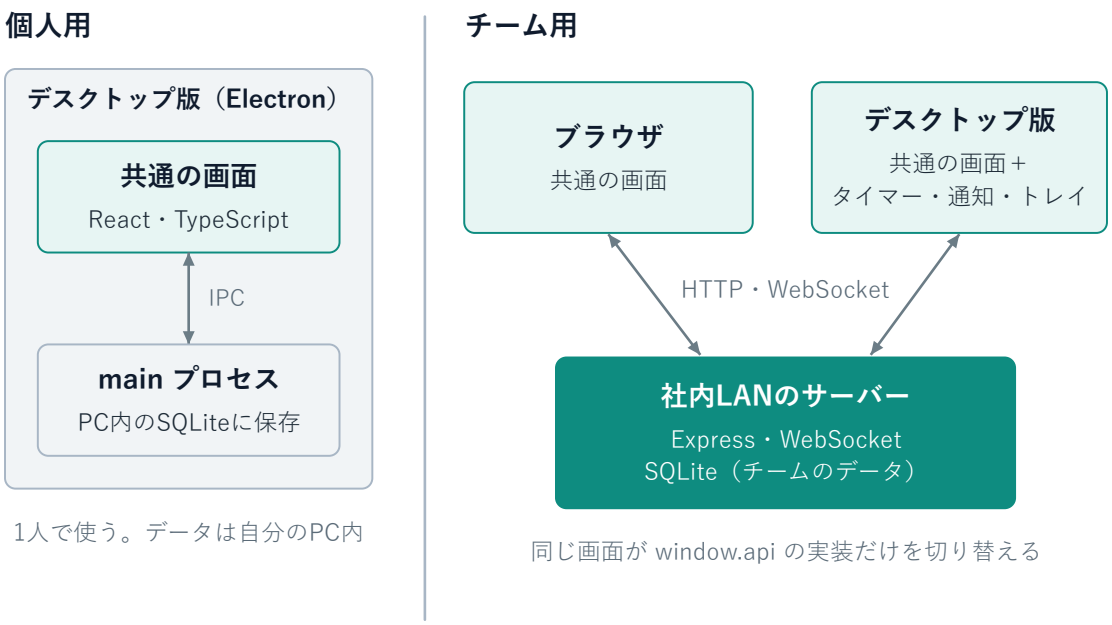


図 2 個人用とチーム用の構成

各構成要素の責務を表 3 に示す。

表 3 構成要素と責務

構成要素	責務	主なファイル
共通の画面	ガント、詳細、報告など、すべての画面。個人用とチーム用で共有する	src/renderer/src/
窓口の定義	画面が呼べる操作の一覧（99 個）。型として、両方の実装に同じ操作を求める	src/preload/index.ts
main プロセス	個人用の DB と、依存関係の再計算。ウィンドウ、トレイ、ショートカットの管理	src/main/

構成要素	責務	主なファイル
Web の窓口	同じ操作を HTTP と WebSocket で実装する	web/lib/client.ts
サーバー	認証、DB への保存、変更の通知、バックアップ、デスクトップ版の配布	server/src/
自動テスト	一時 DB と一時サーバーで、保存・同期・画面の動作を確かめる	tests/、server/tests/

窓口の定義には、タスクやサブタスクの読み書き、進捗ログ、計画、タイマーなど 99 個の操作がある。窓口の型は個人用の定義から作るため、操作を 1 つ足すと、チーム用の実装を書くまで型チェックが通らない。これにより、片方だけに機能がある状態を防いでいる[I2]。

3.3 データの流れと同期

チーム用で、あるメンバーの変更がほかのメンバーの画面に届くまでの流れを図 3 に示す。画面で保存すると、変更した項目だけを HTTP でサーバーへ送る。サーバーはログインを確かめ、DB を更新し、変更履歴を残す。続いて、WebSocket で接続中の全員に「どの種類のデータが変わったか」を知らせる。WebSocket は、サーバーと画面の間で双方向に通知を送るための通信規約である[E5]。知らせを受けた画面は、該当するデータを取り直して表示する。



図 3 チームでの変更が全員の画面に届くまで

通知に本文を載せないのは、受け取った画面が常にサーバーの最新の内容を表示するためである。

複数人が同時に使うと、遅れて届いた古い応答で表示がずれたり、ほかの人の更新を上書きしたりする場合がある。画面は選んでいるタスクと要求の順番を確かめ、古い応答を表示しない。保存では編集した項目だけを送り、同じ項目がほかの人に先に変えられていないかを DB の更新時に確かめる。これらは 10 月 9 日の全体確認で見つけた不具合への対策として加えた（8.3 節）。

サーバーが配信する画面とインストール済みのデスクトップ版は、別々に更新される。そのため、デスクトップ版に足した機能は、画面から呼ぶ前にあるかどうかを確かめる。画面からデスクトップ版へ送る項目を足した場合は、古い画面からは届かない前提で扱う[I2]。

3.4 データの構造

DB は個人用が 15、チーム用が 23 のテーブルで構成する[I6]。主なテーブルを表 4 に示す。チーム用には利用者、ログインの状態、通知、副担当など、複数人で使うためのテーブルを加えている。

表 4 主なテーブル

テーブル	保存する内容	個人用・チーム用
Todos、SubTasks	タスクとサブタスク。日程、進捗、担当、状態	両方
TodoDependencies	依存関係と待機日数	両方
ProgressNotes、ProgressNoteComments	進捗ログと返信	両方
TodoChangeLogs、SubTaskChangeLogs	変更履歴。報告タブの期間中の変化に使う	両方
WorkLogs、RunningState	作業時間の記録と、計測中のタイマー	両方
SubTaskChecks	ToDo リストの項目を周期ごとにチェックした記録	両方
Users、Sessions	利用者とログインの状態	チーム用
Notifications	通知（返信、リアクション、名前の呼びかけ、担当の割り当て、期限、報告のリマインド）	チーム用

3.5 技術の選択

主な技術と、選んだ理由、代わりの案を表 5 に示す。

表 5 技術の選択

技術	選んだ理由（利点）	代わりの案と、選ばなかった点
Electron	デスクトップ版で、トレイ、OS の通知、全体のショートカット、小さなタイマーの窓を使える[E1]	ブラウザだけで作る案。端末の機能を使えず、計測や通知に気付きにくい
React、TypeScript	画面を個人用とチーム用で共有できる。窓口の左右の対応を型で確かめられる	画面を別々に作る案。同じ機能を二度作り、差が生じやすい

技術	選んだ理由（利点）	代わりの案と、選ばなかった点
SQLite	ファイル 1 つで動き、DB 製品の構築と保守が要らない。WAL（Write-Ahead Logging）で読み取りと書き込みが互いを妨げない[E3]	専用の DB サーバーを使う案。6 人程度の規模に対して構築と保守の負担が大きい
Express、ws	Node.js の 1 つのプロセスで、HTTP と WebSocket の両方を扱える	画面が定期的に問い合わせる案。変更の反映が遅れ、通信も増える
bcrypt（bcryptjs）	パスワードをそのまま保存せず、復元しにくい値（ハッシュ）にして保存する	Argon2id を使う案。9.4 節で述べるとおり見直しの候補とする

サーバーは server/の独立したパッケージとして管理している。SQLite を扱う部品（better-sqlite3）の機械語の互換（Application Binary Interface、ABI）が、Electron 用と Node.js 用で異なるためである[I2]。

4. チームの仕事を見える化する仕組み

この章では、G1「チームの仕事が見える」に対応する機能を説明する。

4.1 ガントチャート

ガントの画面の例を図 4 に示す。タスクとサブタスクを日程のバーで並べ、担当と進捗をバーの上に表示する。表示の単位は日・週・月・年から選べ、土日・祝日の列と今日の列を色で示す。チーム用では、担当者別に並べ替えることもできる[I1]。

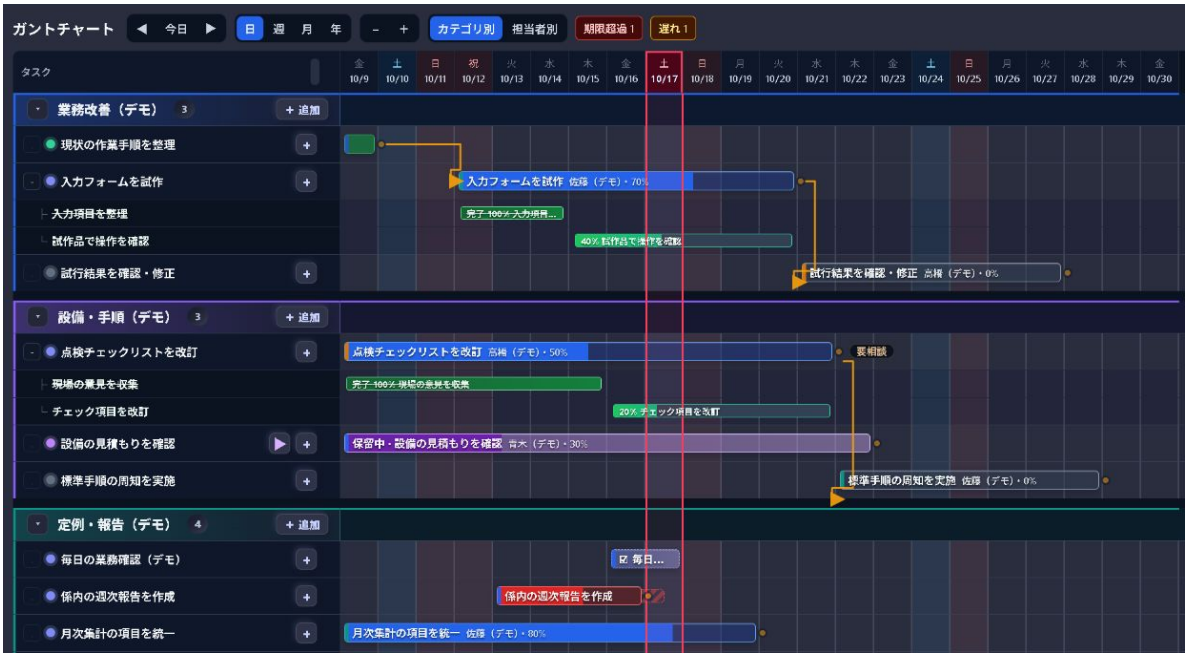


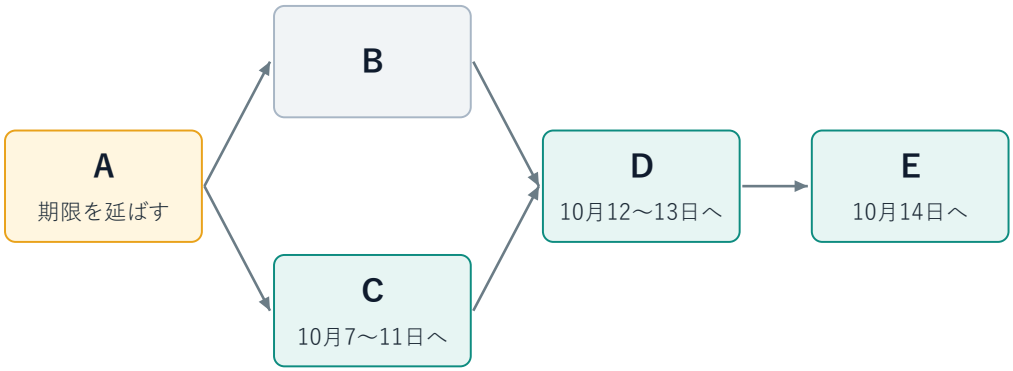
図 4 ガントの画面（デモのデータ）

バーの色は予定に対する遅れを示す。期間のうち経過した日数の割合を「予定の進捗」とし、実際の進捗率との差を日数に換算する。差が設定の日数（既定は 2 日）以上なら「遅れ」として橙色にする。期限を過ぎたものは赤、開始前は灰色、それ以外は青で示す。土日・祝日を除いて数える設定もある[I1][I2]。色は入力された日程と進捗から計算した目安であり、遅れの理由は進捗ログや本人の話で確かめる。

4.2 依存関係の再計算

HAKOBI は依存関係として、前の仕事が終わってから次の仕事を始める関係（Finish to Start）を採用した。前後の間に待機日数を置くこともできる。依存関係はガントとタスク詳細のどちらからでも編集できる[I2]。

前の仕事の日程を変えると、後ろに続く仕事の日程を計算し直す。依存関係の追加、待機日数の変更、削除でも同じく再計算する。図 5 は A から B と C に分かれ、D で合流して E へ続く例である。A の期限を延ばすと、C が後ろへずれ、C の後に始める D とその後ろの E も後ろへずれる。



矢印は「前の仕事が終わってから次を始める」（Finish to Start）。待機日数も足せる。
すべての前工程を計算してから後工程を計算するので、合流するDとその後ろのEにも正しく伝わる。

図 5 分岐と合流がある依存関係の再計算

当初の実装は同じタスクを一度しか計算しなかった。そのため、一方の枝を更新する前に合流先の D を計算すると、D が古い日程のまま残った。10 月 9 日の全体確認でこれを見つけ、すべての前工程を計算してから後工程を計算する順番に改めた[I4]。この計算は個人用とチーム用の両方の DB の層にあり、両方を同じ規則で直した。

4.3 チーム画面

チーム画面の例を図 6 に示す。上段の「いま稼働中」には、メンバーがタイマーで計測している仕事を表示する。中段には、期限を過ぎた仕事と、期限まで 3 日以内の仕事を並べる。下段の「メンバーの負荷」には、メンバーごとの進行中・保留中・期限超過の件数と、今日の作業時間を表示する。仕事の偏りや、手助けが必要な案件を話し合う材料になる。



図 6 チーム画面（デモのデータ）

計測中のタイマーは、利用者ごとに 1 つまでとした。

4.4 進捗ログと相談

進捗の画面を図 7 に示す。作業の区切りに、進捗率を更新して短い進捗ログを残す。「ひな形」を押すと「やったこと」「次にやること」「困っていること」が入力欄に入る。大きな変化がなければ、「予定どおり」で一言の報告を投稿できる。



図 7 進捗の画面（デモのデータ）

判断や手助けが必要な進捗ログには「要相談」の印を付ける。ほかのメンバーは返信と 4 種類のリアクションで応えられる。本文に「@名前」と書くと、その人に通知が届く。「@名前」がだれを指すかの判定には、サーバーと画面で同じ規則を使う[I2]。進捗の画面では、前回この画面を見た時刻より後に、ほかの人が書いた投稿と返信を「新着」として示す。

5. 記録を定例と週報に使う仕組み

この章では、G2「記録が一度で済む」に対応する仕組みを説明する。

5.1 書く場所の役割と記録の使い回し

タスクには三つの書く場所がある。表 6 に役割を示す。役割を分けておくと、仕事の目的、今の状況、これまでの経緯を、それぞれ探さずに読める。

表 6 書く場所の役割

書く場所	書く内容	例
説明	仕事の目的や進め方	集計項目の違いを整理し、係で使う様式をそろえる
メモ	今の状況（書き換えて使う）	関係先へ確認中。回答を受けたら様式を確定する
進捗ログ	日付つきで積み上がる報告	10 月 16 日　試作を確認。入力項目の追加を相談したい

図 8 に、記録を使い回す考え方を示す。作業の区切りに一度残した進捗率、進捗ログ、メモ、作業時間を、チームの状況確認、定例での相談、週報の下書きの三つに使う。報告のために同じ内容を書き直す回数を減らすことが、この設計の目的である。

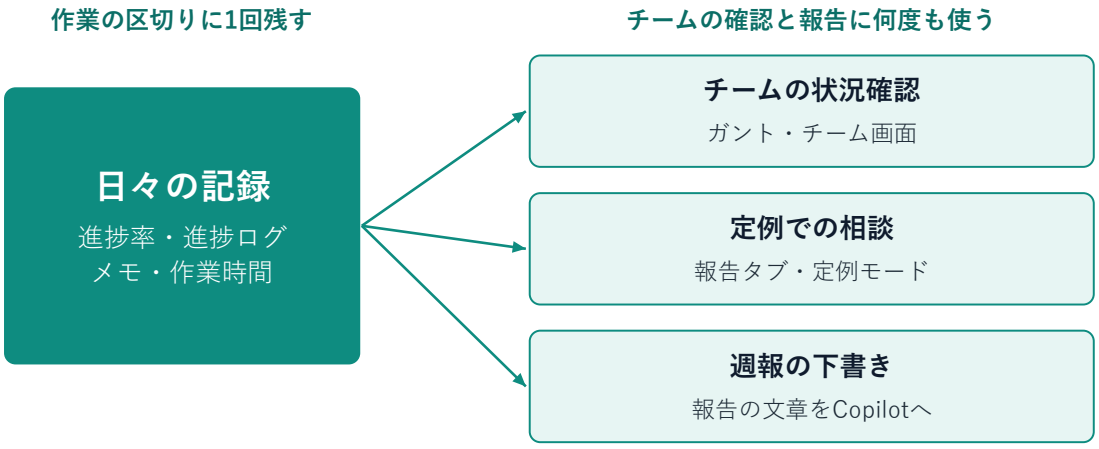


図 8 一度残した記録を確認と報告に使う

5.2 報告タブ

報告タブの例を図 9 に示す。期間（今週、先週、直近 7 日など）を選ぶと、担当者別またはカテゴリ別に、進捗の変化、期限、メモ、進捗ログと返信がまとまって表示される。定例の前に、思い出しながら一覧を作る手間を減らせる。



図 9 報告タブ（デモのデータ）

「期間中の進捗の変化」は、表 4 の変更履歴から計算する。タスクの変更は保存のたびに記録する。サブタスクの変更は、進捗率と期限が変わったときだけ記録する[I2]。

画面の上部には「報告なし」「要相談」「保留中」「期限超過」「期間中に延期」の件数を表示する。「報告なし」は期間内に進捗ログもメモの更新もないタスクを指す。仕事をしていないという意味ではないため、本人に状況を確かめる材料として扱う。

進捗率、期限、メモ、サブタスク、進捗ログは、報告タブから直接更新できる。定例で決まった内容を反映するために、詳細画面へ戻る必要はない。表示中の内容は Markdown の文章と CSV（Comma-Separated Values）で取り出せる。

5.3 定例モード

定例モードの例を図 10 に示す。報告タブを担当者別にして定例モードを開くと、担当者ごとに大きく表示して定例を進められる。持ち時間を選び、経過時間を見ながら、要相談の内容から順に話せる。

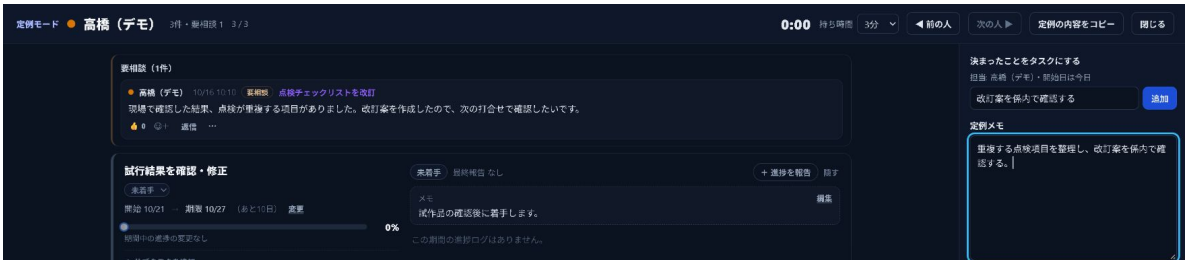


図 10 定例モード（デモのデータ）

追加の確認や対応が決まったら、画面の右側からその場でタスクを作れる。担当は表示中の人、開始日は今日になる。定例メモ、追加したタスク、担当者ごとの時間は、「定例の内容をコピー」でまとめて取り出し、議事録に貼れる。定例メモは報告の画面を離れると消えるため、残すときは先にコピーする。

管理者が定例の曜日を設定すると、報告のリマインドが届く。定例の前日 17 時以降に、直近 7 日に進捗ログもメモの更新もない担当タスクがある人へ、サーバーが 1 日 1 回通知を出す。条件は 1 分ごとに確かめる[I2]。

5.4 週報の下書き

週報の下書きを作る流れを図 11 に示す。報告タブで期間と担当を選び、「Markdown をコピー」を押す。コピーした文章を、Microsoft 365 Copilot で作った週報用のエージェントに貼り付け、下書きを依頼する[E7]。下書きは、担当者、期限、進捗率、相談事項を本人が確かめてから提出する。

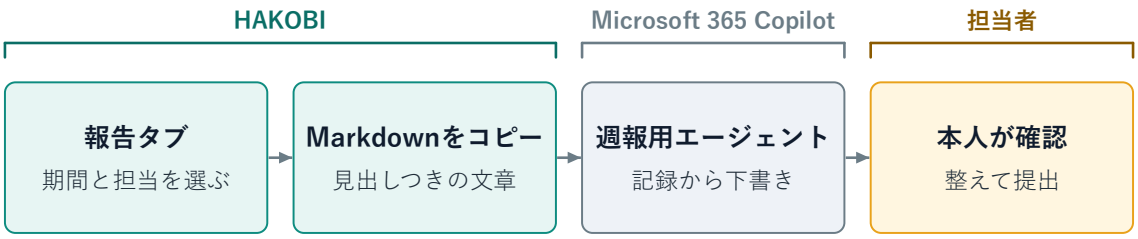


図 11 週報の下書きを作る流れ

HAKOBI から Copilot へ自動で送る機能はない。人が文章をコピーして渡し、内容を確かめる運用である。エージェントに保存する指示の例は付録 C に示す。

5.5 進捗レポートの数え方

一人ひとりの活動をまとめる進捗レポートは、「担当者」ではなく「実際に操作した人」で数える。チーム用では、変更履歴からタスクを完了にした人を求め、その人の完了として数える。追加したサ

ブタスクは、作った人で数える。タスク別のレポートには、集計の対象者が書いた進捗ログと返信だけを載せ、副担当などほかの人の記録を混ぜない[I2]。

6. 一人ひとりの仕事を進める機能

この章では、G3「自分の仕事を進めやすい」に対応する機能を説明する。

6.1 今日の計画と作業時間

計画の画面を図 12 に示す。今日取り組む仕事を時間割に並べ、15 分単位で調整できる。前日に終わらなかった仕事は翌日に繰り越せる。予定が変わったときは、後ろの予定をまとめて移せる[I1]。



図 12 計画の画面（デモのデータ）

タスクにひも付けてタイマーで計測すると、作業時間が記録に残る。計画は予定、作業ログは実際に記録した時間として区別する。回答待ちなどでタスクを保留にすると、計測中のタイマーを止めて作業ログを保存する。チーム用では、同じタスクを計測しているメンバー全員のタイマーを止める。保留中のタスクは、今日の作業の候補と繰り越しの対象から外す[I1]。

6.2 繰り返す仕事と ToDo リスト

繰り返す仕事には二つの形を用意した。週次の資料作成のような仕事は、通常のタスクに繰り返しを設定する。完了すると次回のタスクを作り、土日・祝日を飛ばす設定もできる。

毎朝の点検のように同じ項目を毎回チェックする仕事には、ToDo リストを使う。ToDo リストの画面の例を図 13 に示す。項目ごとに毎日・平日・毎週・毎月の周期を選ぶと、周期が変わったときに未チェックに戻る。チェックの記録は周期ごとに 1 件ずつ残る（表 4 の SubTaskChecks）。

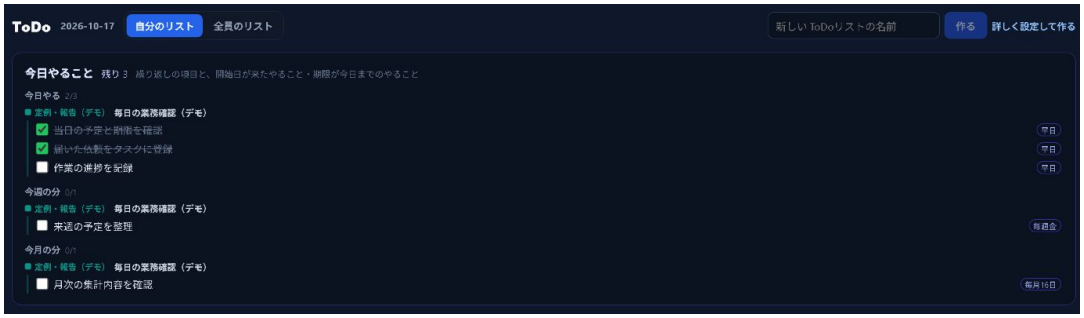


図 13 ToDo リストの今日やること（デモのデータ）

報告タブでは終わった周期の実施状況（例：昨日まで 18/20 回）と、今の周期のチェック状況（例：今日 1/5）を分けて表示する。今日を含む周期をすでに実施した回に数えると、作ったばかりのリストが 1 件のチェックで 100%になってしまう。これを避けるため、今の周期は実施の回数に入れず、割合（%）は終わった周期が 5 回以上になってから表示する[I2]。

6.3 検索・移動と動作の軽さ

Ctrl＋K で移動・操作のパレットを開くと、タスク名、カテゴリ、担当者からタスクを探し、画面を切り替えられる。表示中の画面と選んだタスクは URL（Uniform Resource Locator）に反映するため、タスクのリンクをチャットに貼って共有できる。チーム用では、受け取った人も同じサーバーにログインすれば、そのタスクを開ける。

動作を軽くするため、ガント、一覧、詳細以外の画面は必要になったときに読み込む。計測中の経過秒数は表示する部品の中だけで数え、画面全体を毎秒描き直さない。ガントと一覧の行は、内容が変わらなければ描き直さない[I2]。

6.4 デスクトップ版のチーム接続

チーム接続のデスクトップ版には、ブラウザ版にない端末の機能を加えた。小さなタイマーの窓、進捗の素早い投稿、トレイとタスクバーからの計測の開始・切り替え・停止、OS の通知と未読バッジである[I1]。

通知を出すかどうかは、非表示で動かしているメイン画面が判定し、結果をデスクトップ版の本体へ渡す。本体はサーバーに直接問い合わせない。新着かどうかは端末に保存した通知済みの記録で判定し、初めて接続したときには、既存の通知をまとめて出さないようにした[I2]。

7. 運用と安全の仕組み

この章では、G5「社内で軽く運用できる」に対応する仕組みを説明する。

7.1 アカウントと認証

チーム用のアカウントは管理者が発行し、利用者が自分で登録する仕組みは設けない。パスワードは 6 文字以上とし、bcrypt でハッシュにして保存する。ログインの状態は Cookie で保ち、有効期限の既定は 30 日である[I2]。各自のパスワードは設定の画面から変更できる。

管理者がメンバーを無効にすると、計測中のタイマーを止めて記録し、ログインと同期の接続を終了する。メンバーを削除するときは、担当を外し、作業ログ、投稿、変更履歴は残す。削除したログイン ID（Identifier）は再び使えない[I1]。

7.2 社内 LAN での運用

サーバーは社内 LAN の PC に置き、Windows サービスとして登録して常時動かせる。既定のポートは 4577 である。グループごとにデータを分ける場合は、グループごとにサーバーのプロセスを立て、ポート、データの保存先、Cookie の名前を分ける。ポート番号を変えるだけでは DB は分かれない[I3]。

7.3 バックアップ

サーバーは起動時と 1 分ごとの点検で、その日のバックアップがなければ取る。バックアップは 1 日 1 つで、既定では 14 日分を残して古いものから消す。管理者は設定の画面から、現在の DB を 1 つのファイルとしてダウンロードできる。SQLite は WAL で書き込むため、動いている間に DB のファイルだけをコピーすると、直近の更新が欠けることがある。自動バックアップとダウンロードには、SQLite のバックアップ機能を使う[I3]。復元は画面からは行わず、サーバーを止めてファイルを置き換える手順を運用ガイドに定めた。

7.4 デスクトップ版の配布

デスクトップ版のインストーラーは、サーバーの「/download」ページから入手できる。このページはログインなしで開ける。サーバーは、配布フォルダの `latest.yml` に書かれたインストーラー 1 つだけを配る。ファイル名、サイズ、SHA-512（Secure Hash Algorithm 512-bit）の値が一致しないものは配らない[I2]。

公開は専用のコマンドで行い、インストーラーを先に置き、`latest.yml` を最後に置き換える。コピーの途中のファイルが配られることはない。新しいバージョンを公開するとデスクトップ版の画面にお知らせが出るが、インストールは各自が行う[I3]。

7.5 個人用データの取り込み

個人用で作ったタスクは、管理者がチーム用へ取り込める。取り込み先のメンバーを選び、まず「ドライラン（確認のみ）」で件数を確かめてから実行する。タスク、サブタスク、依存関係、作業ログ、予定、進捗ログと返信、変更履歴を引き継ぐ。取り込みは個人用からチーム用へのコピーであり、再び実行しても取り込み済みのデータは更新しない。取り込んだ後も、個人用とチーム用の DB は同期しない。

8. 品質の確認

8.1 確認の方法

品質は型チェック、ビルド、自動テストの三つで確かめた。

型チェックでは、TypeScript のコンパイラーで、main プロセス、共通の画面、Web の窓口、サーバーの 4 つを検査する。窓口の左右の対応（3.2 節）もこの段階で確かめられる。ビルドでは、Web の画面を一時フォルダへ出力し、本番用の組み立てが成功することを確かめる。

自動テストは Node.js で実行するスクリプト 36 ファイルと、Node.js のテストランナーで実行するテスト 12 ファイルからなる。テストはそれぞれ独立した一時 DB、一時フォルダ、一時サーバーを使い、運用中の DB やサービスに触れない。画面の動作はウィンドウを表示しない状態で実際の Electron を起動して確かめる[I3]。対象は保存先の変更、削除、依存関係の再計算、同時編集、タイマー、認証、取り込み、配布、進捗ログ、保留、ToDo リスト、ガントの表示位置などである。

8.2 結果

表 7 に、2026 年 10 月 17 日に実行した結果を示す。自動テストは、最初の実行で見つけた問題を直したあとに実行し直した。環境は Windows 11、Node.js 22.16.0、Electron 29.4.6 である。所要時間は秒単位で計った[I6]。

表 7 品質確認の結果（2026 年 10 月 17 日）

確認	件数	結果
型チェック	4 種	すべて成功（エラー 0 件）
Web のビルド	1 回	成功（所要時間 5 秒）
自動テスト（最初の実行）	15 コマンド	14 コマンドが合格。test:quality の 1 テストが時間切れ（下記）
自動テスト（修正後の実行）	15 コマンド	すべて合格。最長は test:hakobi の 69 秒

最初の実行で時間切れとなったのは、個人用データの保存先の変更や削除を確かめる `local-data-regressions` である。11 項目すべての合格を出力した後、テスト用に起動したプロセスが 60 秒の制限時間内に終了しなかった。2 回実行して、2 回とも同じ結果であった。

検査の後に残っていた処理を調べると、1 分ごとに動くタイマーが 1 つ残っていた。10 月 14 日に ToDo リストのために加えた、日付が変わったら繰り返しの項目を未チェックに戻す点検のタイマーである（`src/main/ipc.ts`）。Node.js は、動いているタイマーがあるあいだプロセスを終了しない。テストはこの点検を登録する処理を呼ぶため、検査が終わってもプロセスが終わらなかった。

このタイマーを、プロセスの終了を引き止めない設定（`unref`）に改めた。アプリはウィンドウとトレイで動き続けるため、アプリが動いているあいだは点検をこれまでどおり行う。修正後は `local-data-regressions` が自然に終了し、15 コマンドすべてが合格した。

8.3 全体確認で見つけた不具合と対策

10 月 9 日に、個人用、共通の画面、サーバーの管理機能まで範囲を広げて全体を確認し、13 件の不具合を再現した[I4]。表 8 に区分ごとの内容を示す。P1 はデータの欠落、誤った更新、サーバーの停止につながる不具合、P2 は日程の計算、操作、同期、取り込みの不具合である。

表 8 2026 年 10 月 9 日の全体確認で見つけた不具合

区分	件数	主な内容
P1	6 件	保存先の変更で直近の更新がコピーされない。「自動削除しない」の設定で削除する。自動削除で作業ログだけが消える。特定の Cookie でサーバーが停止する。遅れて届いた応答で別のタスクのサブタスクを表示・変更できる。ほかのメンバーの更新を古い値へ戻す
P2	7 件	分岐と合流がある依存関係を正しく再計算できない。別の窓からの移動で未保存の確認を飛ばす。保存の失敗が画面に出ない。無効にしたメンバーの計測が続く。メンバーの追加が同期されない。取り込みで進捗の本文と集計外の設定が欠ける

13 件はすべて修正した。修正後の再レビューで見つけた問題も追加で直した。保存先の変更、失敗時の処理、古い応答、同時編集などの主な再現条件は、自動テストに加えた[I4]。この確認で、既存のテストが通っていても、分岐と合流、保存先の変更、古い応答、同時編集の不具合は見つけられていなかったことが分かった[I4]。

9. 考察

9.1 目指した姿ごとの到達点

表 9 に、目指した五つの姿ごとに、実現したことと、それを確かめた方法を示す。

表 9 目指した姿ごとの到達点

目指した姿	実現したこと	確かめた方法
G1 チームの仕事が見える	ガント、依存関係の再計算、遅れの色、チーム画面、変更の即時通知、進捗ログと要相談	依存関係と同時編集の自動テスト、全体確認での再現と修正
G2 記録が一度で済む	書く場所の役割分担、報告タブ、定例モード、報告のリマインド、Markdown の出力	進捗ログ・保留・リマインドの自動テスト
G3 自分の仕事を進めやすい	計画、タイマーと作業ログ、繰り返し、ToDo リスト、パレット、デスクトップ版の通知	タイマー・ToDo リスト・ガント表示の自動テスト、実 Electron の確認
G4 個人でもチームでも同じ画面	共通の画面と窓口の型、個人用データの取り込み	4 種の型チェック、取り込みの自動テスト
G5 社内で軽く運用できる	管理者によるアカウント発行、グループごとの分離、自動バックアップ、検証つきの配布	認証・バックアップ・配布の自動テスト

五つの目標に対応する機能を実装し、自動テストと型チェックで動作を確かめた。以降では設計上の利点と制約を考察する。

9.2 画面を共通にしたことの利点と代償

画面を共通にしたことで、機能を一度作れば個人用とチーム用の両方で使える。利用者は個人用で使い方を覚え、そのままチーム用へ移れる。窓口の左右の対応は型で強制されるため、片方だけの機能が残りにくい。

一方で、代償もある。データの型は、main プロセスと画面にそれぞれ定義があり、同じ構造に保つ必要がある。依存関係の再計算のように、同じ規則を個人用とチーム用の両方の DB の層に持つ処理もある。8.3 節の再計算の不具合は、両方で同時に直す必要があった。新旧の画面が混ざる前提の設計も欠かせない。これらは変更のたびに両方を確かめるテストで補っている。

9.3 SQLite と 1 台のサーバーの利点と限界

SQLite を選んだことで、別の DB 製品を構築・保守する負担を省けた。データはファイルとして扱えるため、バックアップと復元の手順も単純になる。SQLite では、1 つの DB ファイルに同時に書き込めるのは 1 つの処理だけである[E4]。WAL により読み取りと書き込みは互いを妨げないが[E3]、書き込みが非常に多い使い方には向かない。HAKOBI は 1 つのサーバーで 6 人程度の利用を想定しており、この範囲では書き込みの集中は起きにくいと考える。規模を大きく超える場合は、グループごとにサーバーを分けるか、DB 製品の利用を検討する。

サーバーが止まると、チーム用ではタスクや進捗を保存できない。進捗の下書きは端末に残り、再接続した後に投稿できる[I3]。復旧に備え、サーバーを Windows サービスとして動かし、1 日 1 回のバックアップを取る。

9.4 安全性についての判断

チーム用は社内 LAN での利用を前提とし、通信は HTTP で行う。HTTP は通信を暗号化しないため、LAN の中で通信を盗み見られた場合、内容やログインの Cookie が読まれるおそれがある。社内 LAN の外へは公開しない前提で現在の構成としたが、HTTPS（Hypertext Transfer Protocol Secure）による通信の暗号化は今後の課題とする。

パスワードの保存には bcrypt を使い、計算の重さを表す係数は 10 とした。OWASP（Open Worldwide Application Security Project）の指針は、新しいシステムには Argon2id を勧め、bcrypt を使う既存のシス

テムには係数 10 以上を求めている[E2]。HAKOBI の設定はこの下限を満たすが、方式とパスワードの最小の長さ（6 文字）は見直しの候補とする。パスワードを変えたときに既存のログインを終了するかどうかも、まだ決めていない[I4]。

カテゴリの「集計外」は集計の対象を分ける設定であり、ほかの人から見えなくする設定ではない。チームのタスクは、メンバー全員が見られる。見せたくない情報を書く場所としては使わないよう、利用者に伝える必要がある。

9.5 効果の確かめ方

本レポートの時点では、導入による効果を測っていない。聞き直しや書き直しの手間が減ったかどうかは、本レポートの結果からは言えない。効果を確かめるため、表 10 の指標を提案する。いずれも HAKOBI の機能で記録できるか、導入の前後で比べられる値である。

表 10 効果を確かめる指標の案

指標	測り方	期待する変化
定例の所要時間	定例モードの担当者ごとの時間（「定例の内容をコピー」で出力される）	準備済みの記録を使うことで短くなる
週報の作成にかかる時間	週報の作成をタイマーで計測し、導入の前後で比べる	下書きを使うことで短くなる
報告なしのタスクの割合	報告タブの「報告なし」の件数を週ごとに記録する	作業の区切りに記録する習慣が定着すれば減る
期限超過の件数	チーム画面と報告タブの「期限超過」を週ごとに記録する	遅れに早く気付けば減る

10. 結論と今後の展望

10.1 結論

HAKOBI はチームの予定・担当・進捗を同じ画面で確かめ、作業の区切りに残した記録を定例と週報に使い回すための業務管理ツールである。

個人用とチーム用は同じ画面を共有し、データの窓口だけを切り替える構成とした。チーム用では、変更を WebSocket で知らせて全員の画面を最新に保ち、古い応答や同時編集による上書きを防いだ。

ガントと依存関係の再計算、チーム画面、進捗ログと要相談で仕事を見える化し、報告タブ、定例モード、報告のリマインド、Markdown の出力で記録を定例と週報につないだ。計画、タイマー、ToDo リスト、デスクトップ版の通知で、一人ひとりの日常業務も支えた。運用では、管理者によるアカウントの発行、自動バックアップ、検証つきの配布を用意した。

型チェックとビルドはすべて成功した。自動テストは、時間切れの原因となったタイマーを直したあと、15 コマンドすべてが合格した。10 月 9 日の全体確認で見つけた 13 件の不具合は修正した。以上から、目指した五つの姿に対応する機能を実装できたと判断する。

全体確認からは、既存の自動テストが通っていても、分岐と合流がある依存関係、保存先の変更、古い応答、同時編集の不具合が残りうることが明らかになった。個人用とチーム用で同じ規則を持つ処理と、複数人が同時に使う場面は、再現条件を決めたテストで確かめる必要がある。また、動き続けるアプリに加えた定期処理のタイマーは、同じコードを読み込むテストのプロセスの終了を妨げることがある（8.2 節）。

10.2 業務への貢献

チームの仕事の状況を、本人に聞かずに画面で確かめられるようになった。定例と週報の材料を日々の記録から取り出せるため、報告のための書き直しを減らす下地ができた。週報の下書きを Copilot で作る流れは、日々の記録を生成 AI（Artificial Intelligence）の入力として使う例でもある。個人用から始めてチーム用へ移れること、グループごとにサーバーを分けられることから、ほかの係やグループへ広げる際の基盤としても使える。

10.3 今後の展望

次の順に進める。

1. 表 10 の指標で効果を測る。定例の所要時間、週報の作成時間、報告なしの割合、期限超過の件数を、導入の前後で比べる。
2. 通信の暗号化（HTTPS）、パスワードの方式と最小の長さ、パスワード変更時のログインの扱いを見直す。
3. ほかの係やグループへ広げるため、サーバーの準備、アカウントの発行、個人用データの取り込みの手順を整える。

参考文献

内部資料

- [I1] `README.md`, 「HAKOBI」, 2026 年 10 月 17 日参照。
- [I2] `CLAUDE.md` および `AGENTS.md`, 開発者向けの設計メモ（技術構成、実装メモ）, 2026 年 10 月 17 日参照。
- [I3] `docs/HAKOBI.md`, 「HAKOBI のチーム接続と運用」, 2026 年 10 月 17 日参照。
- [I4] `docs/HAKOBI_FULL_REVIEW_2026-10-09.md`, 「HAKOBI 全体確認（2026 年 10 月 9 日）」。
- [I5] Git の履歴（`main`、2026 年 3 月 4 日〜10 月 17 日、コミット `f9c1bc6` まで）。
- [I6] `docs/hakobi-report/evidence/VALIDATION_20261017.md`, 本レポートの検証記録。
- [I7] `docs/hakobi-introduction/report-internal.md`, 「チームの仕事が見える業務管理ツール HAKOBI を作りました」（社内レポート版）, 2026 年 10 月 17 日。画面の図の元にしたデモの撮影も同じ日に行った。

外部資料

- [E1] Electron, “Process Model,” https://www.electronjs.org/docs/latest/tutorial/process-model, 2026 年 10 月 17 日参照。
- [E2] OWASP Foundation, “Password Storage Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026 年 10 月 17 日参照。
- [E3] SQLite Project, “Write-Ahead Logging,” https://www.sqlite.org/wal.html, 2026 年 10 月 17 日参照。
- [E4] SQLite Project, “Appropriate Uses For SQLite,” https://www.sqlite.org/whentouse.html, 2026 年 10 月 17 日参照。
- [E5] IETF, RFC 6455, “The WebSocket Protocol,” 2011, https://www.rfc-editor.org/info/rfc6455.
- [E6] React, https://react.dev/ ; Express, https://expressjs.com/ ; better-sqlite3, https://github.com/WiseLibs/better-sqlite3, 2026 年 10 月 17 日参照。
- [E7] Microsoft, 「独自のエージェントを構築する」, https://support.microsoft.com/ja-jp/microsoft-365-copilot/build-your-own-agent-with-microsoft-365-copilot ; “Build agents in Agent Builder,” https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/agent-builder-build-agents, 2026 年 10 月 17 日参照。

付録 A 用語と略語

本レポートで使う用語と略語を表 A1 に示す。略語は、概要と本文のそれぞれで初めて使う箇所にも正式名称を添えた。

表 A1 本レポートで使う用語と略語

用語・略語	意味
個人用・チーム用	1 人で使うデスクトップ版と、社内 LAN のサーバーを複数人で使う形
チーム接続	デスクトップ版からチーム用のサーバーへ接続して使うこと
ガントチャート	仕事を横棒で時間軸に並べ、開始日、期限、進み具合を示す図
進捗ログ	仕事ごとに日付つきで積み上がる短い報告
要相談	判断や手助けが必要な進捗ログに付ける印
Finish to Start	前の仕事が終わってから次の仕事を始める依存関係
LAN（Local Area Network）	社内など限られた範囲のネットワーク
PC（Personal Computer）	利用者の端末
DB（Database）	データを保存し、検索や更新を行う仕組み
SQLite	ファイル 1 つで動く DB。別のサーバーを必要としない
WAL（Write-Ahead Logging）	SQLite の書き込み方式。読み取りと書き込みが互いを妨げない
ABI（Application Binary Interface）	機械語の部品どうしの互換の取り決め
IPC（Inter-Process Communication）	同じ PC の中のプロセスどうしの通信
HTTP／HTTPS	Web の通信規約。HTTPS（Hypertext Transfer Protocol Secure）は暗号化した HTTP
WebSocket	サーバーと画面の間で双方向に通知を送る通信規約
API（Application Programming Interface）	プログラムどうしが要求と応答をやり取りする窓口。本文では window.api を「窓口」と呼ぶ
URL（Uniform Resource Locator）	画面やファイルの場所を示す文字列
CSV（Comma-Separated Values）	表計算ソフトで開ける、カンマ区切りの表形式ファイル
OS（Operating System）	Windows など、PC の基本ソフト

用語・略語	意味
PR（Pull Request）	変更を本流に取り込む前の確認の単位
SHA-512（Secure Hash Algorithm 512-bit）	データの内容から 512 ビットの要約値を求める方式。ファイルの一致を確かめる
OWASP（Open Worldwide Application Security Project）	Web の安全性に関する指針を公開する団体
DX（Digital Transformation）	デジタル技術で業務の進め方を変えること
AI（Artificial Intelligence）	人工知能。本文の生成 AI は、指示に応じて文章などを作るもの
ID（Identifier）	利用者などを区別するための名前や番号
Markdown	見出しや箇条書きを記号で書く文章の形式

付録 B 主な既定値

本文で触れた HAKOBI の主な既定値を表 B1 に示す。値は 2026 年 10 月 17 日時点のコードと運用文書による[I2][I3]。

表 B1 HAKOBI の主な既定値

項目	既定値	補足
サーバーのポート	4577	設定ファイルで変えられる
ログインの有効期限	30 日	設定ファイルで変えられる
パスワードの最小の長さ	6 文字	9.4 節で見直しの候補とした
bcrypt の係数	10	計算の重さを表す
自動バックアップ	1 日 1 つ、14 日分	残す数は設定で変えられる。0 で止める
期限が近い仕事	期限まで 3 日以内	チーム画面の「まもなく期限」
遅れの判定	予定より 2 日分以上の遅れ	表示設定で変えられる
報告のリマインド	定例の前日 17 時以降	直近 7 日に報告がない担当タスクが対象
ToDo リストの実施の割合	終わった周期が 5 回以上	今の周期は数えない
計画の時間割	15 分単位	後ろの予定をまとめて移せる

付録 C 週報用エージェントの指示の例

Microsoft 365 Copilot で週報用のエージェントを作るときに、指示欄に保存する文の例を示す[E7]。係で使う週報の書式が決まっていれば、見出しや長さを合わせる。

```
あなたは係内の週報の下書きを作成する担当です。
今回の依頼に貼り付けられた HAKOBI の報告 Markdown と、依頼者が明示した補足だけを根拠にしてください。

最初に対象期間と担当範囲を確認してください。不明な場合は確認を求め、推測で埋めないでください。
業務ごとに内容をまとめ、次の見出しで簡潔な日本語の週報を作成してください。
1. 今週の主な実施内容
2. 継続中の仕事と現在の状況
3. 相談事項と保留中の仕事
4. 次に行うこと
5. 提出前に確認すること

担当者、期限、進捗率、保留の理由は、記録の内容を保ってください。
期間中の進捗の変化と現在の進捗率を混同しないでください。
要相談の記録と返信を読み、解決したと確認できない内容を解決済みにしないでください。
「期間内の記録なし」を「実施していない」という意味に受け取らないでください。
次の作業は記録に明示されたものだけを挙げ、来週の予定や成果を創作しないでください。
足りない情報や食い違いは「提出前に確認すること」に分けてください。
同じ内容は重複させず、業務名と具体的な動きが伝わる文章にしてください。
```